

Онбординг по REST сервису reminder

Понятное объяснение архитектуры классов методов и тестов

Модуль reminder

Состояние исходников на 4 сентября 2026 года

Как пользоваться документом

Этот документ помогает понять, как устроен модуль reminder, не читая весь проект подряд. Сервис принимает REST-запросы, хранит напоминания, планирует отправку писем и следит, чтобы повторный запрос случайно не создал дубликат.

В каталоге разобраны 37 классов и интерфейсов рабочего кода и 11 тестовых классов. Для каждого типа указано, зачем он нужен. Для каждого метода приведена сигнатура из кода и простое объяснение его действий. Методы, которые Java, Lombok или Spring создают автоматически, перечислены отдельно.

Рекомендуемый порядок чтения в первый день

Сначала прочитайте главы «Короткий словарь», «Архитектура» и «Сквозные сценарии». Они дают общую картину без привязки к отдельным классам.

Затем посмотрите REST-контракт и правила работы со временем, JWT и idempotency key.

После этого разберите QuartzReminderService. В нём находится основная последовательность создания, изменения, поиска и удаления напоминаний.

Каталог рабочего кода используйте как справочник: найдите нужный класс, а затем откройте указанный исходный файл.

Перед изменением поведения найдите соответствующий тест в главе «Тесты». Там указано, какое правило проекта он защищает.

Навигация

Архитектура и границы сервиса

REST контракт

Сквозные сценарии

Каталог рабочего кода

Данные и конфигурация

Тесты

Запуск и диагностика

Ограничения и технический долг

Короткий словарь

Эти определения нужны только для первого знакомства. Дальше документ использует те же слова в описаниях классов и методов.

Слои приложения

- REST endpoint — сочетание HTTP-метода и адреса, по которому клиент вызывает операцию сервиса. Например, POST /api/v1/reminder создаёт напоминание.

- Controller — класс на границе HTTP. Он читает URL, заголовки, JWT и JSON, проверяет их форму, вызывает сервис и собирает HTTP-ответ.
- Payload — объект с данными входного JSON. DTO — объект с данными, которые сервис возвращает клиенту.
- HTTP status — числовой результат запроса. Например, 200 означает успех, 201 — создание, 204 — успех без тела, 400 — ошибку запроса, 401 — отсутствие аутентификации, 404 — объект не найден, 409 — конфликт и 500 — внутреннюю ошибку сервиса.
- Service — класс с правилами приложения: кто имеет доступ, какое время допустимо, что сохранить и какую задачу запланировать.
- Entity — Java-объект, связанный с записью таблицы. JPA — стандарт такого связывания, а Hibernate — библиотека, которая выполняет его в этом проекте.
- Repository — интерфейс доступа к БД. Spring создаёт его реализацию и строит многие запросы прямо из названий методов.

Spring и Java

- Bean — объект, который создаёт Spring и передаёт другим объектам как зависимость. Такой способ связи называется dependency injection.
- Component scan — поиск классов Spring по аннотациям. Autoconfiguration — автоматическое создание стандартных компонентов на основании библиотек и application.yaml.
- Record — компактный класс данных Java. Он автоматически получает конструктор, методы чтения полей, equals(), hashCode() и toString().
- Lombok — библиотека, которая генерирует повторяющийся Java-код во время компиляции, например конструкторы, getters, setters, builder и logger.
- public, protected и private — уровни доступности метода. static означает, что метод принадлежит классу, а не отдельному объекту. void означает, что метод не возвращает значение.
- Runtime exception — ошибка, которую метод не обязан объявлять в сигнатуре. В REST-слое GlobalExceptionHandler превращает известные exceptions в HTTP-ответы.
- Mock — заменитель настоящей зависимости в тесте. Он не подключается к внешней системе и позволяет проверить, с какими аргументами был вызван метод.
- Unit-тест проверяет один класс изолированно. MVC-тест проверяет HTTP-слой без настоящей БД. Интеграционный тест поднимает несколько реальных частей системы и проверяет их совместную работу.
- Testcontainers — библиотека, которая временно запускает PostgreSQL или другой сервис в Docker на время интеграционного теста.

Данные безопасность и фоновые задачи

- Transaction — группа изменений БД. Commit подтверждает всю группу, rollback отменяет её при ошибке.
- Idempotency — защита от повторного выполнения одного запроса. Клиент повторяет тот же Idempotency-Key и получает существующий результат вместо дубликата.

- JWT — подписанный токен с данными пользователя. Поле sub содержит userId, issuer указывает, кто выпустил токен, а audience — для какого сервиса он предназначен.
- Quartz Job — код фоновой задачи. Trigger хранит условие и время её запуска. JobDataMap хранит данные, которые понадобятся задаче.
- SMTP — протокол отправки email. Retry — повторная попытка после временной ошибки.
- Liquibase — инструмент последовательного изменения схемы БД. Каждый changeset является одним зарегистрированным шагом, например созданием таблицы или индекса.
- Actuator — служебные HTTP-адреса Spring Boot для проверки состояния и получения метрик приложения.
- Page и Pageable — содержимое одной страницы и параметры её получения. page задаёт номер, size — размер, sort — порядок.
- Optional — контейнер Java, который либо содержит найденное значение, либо явно показывает, что результата нет.

Архитектура и границы сервиса

reminder — отдельный Spring Boot сервис на Java 21. Он предоставляет HTTP API для напоминаний, хранит данные в PostgreSQL и передаёт отправку писем планировщику Quartz. Вход пользователя выполняет другой сервис — auth-service. Модуль reminder только проверяет полученный JWT и использует данные из него.

HTTP- запрос
 → проверка JWT
 → REST- контроллер
 → бизнес- сервис ReminderService
 → PostgreSQL для данных
 → Quartz для запуска задачи в нужное время
 → Email Service для отправки письма через SMTP

Слой	Основные классы	Что происходит
Получение запроса	RemindersRestController ReminderRestController	Читаются URL, JSON, заголовки и данные пользователя из JWT. Формируется HTTP-ответ.
Правила приложения	ReminderService QuartzReminderService	Проверяются владелец, время и повтор запроса. Решается, что сохранить и какую задачу запланировать.
Работа с данными	ReminderRepository IdempotencyRecordRepository	Напоминания и служебные записи читаются и изменяются в PostgreSQL.
Запуск писем	QuartzEmailScheduler EmailReminderJob	Quartz хранит время запуска, выполняет отправку и назначает повтор после ошибки.
Инфраструктура	DefaultEmailService SecurityConfig Liquibase Actuator	SMTP отправляет письмо, JWT защищает API, Liquibase создаёт таблицы, Actuator сообщает состояние сервиса.

Клиент не передаёт userId отдельным параметром. Контроллер берёт его из поля sub проверенного JWT. Затем сервис добавляет этот userId во все запросы к БД. Поэтому пользователь не может прочитать, изменить или удалить чужое напоминание, просто подставив

его id. Для безопасности чужое напоминание выглядит так же, как несуществующее: API возвращает 404.

REST контракт

Все адреса API начинаются с `/api/v1/reminder`. Swagger и проверки состояния сервиса доступны без токена. Остальные запросы должны содержать Bearer JWT. Для создания и обновления напоминания в токене также ожидается email: он попадёт в будущую задачу отправки письма.

HTTP	Путь	Метод Java	Результат
POST	<code>/api/v1/reminder</code>	<code>createReminder</code>	201, адрес и созданное напоминание
PATCH	<code>/api/v1/reminder/{id}</code>	<code>updateReminder</code>	200 и обновлённое напоминание
DELETE	<code>/api/v1/reminder/{id}</code>	<code>deleteReminder</code>	204 без тела ответа
GET	<code>/api/v1/reminder/sort/title</code>	<code>findAllSortedByTitle</code>	200 и страница данных
GET	<code>/api/v1/reminder/sort/date</code>	<code>findAllSortedByDate</code>	200 и страница данных
GET	<code>/api/v1/reminder?title=...</code>	<code>findRemindersByTitle</code>	200 и найденные совпадения
GET	<code>/api/v1/reminder/search-by-date</code>	<code>findRemindersByDate</code>	200 и напоминания за локальный день
GET	<code>/api/v1/reminder/filter/date</code>	<code>findRemindersByDateRange</code>	200 и напоминания между двумя моментами
POST	<code>/api/v1/reminder/addsomereminders</code>	<code>addSomeReminders</code>	200 и текст Reminders added

Заголовок Idempotency-Key обязателен для создания и обновления. Это строка длиной до 255 символов, которая позволяет безопасно повторить запрос после сетевой ошибки. Если тот же пользователь отправит запрос с тем же ключом ещё раз, сервис постарается вернуть уже полученный результат, а не выполнить операцию повторно.

Заголовок X-Timezone указывать необязательно. Если его нет или он пустой, сервис использует Asia/Almaty. Если заголовок указан, но название часового пояса неизвестно Java, API возвращает ошибку 400.

Во входном JSON поле remind имеет тип LocalDateTime — дату и время без часового пояса. В ответе оно имеет тип OffsetDateTime — дату и время вместе со смещением относительно UTC. Так клиент видит, какой именно момент был сохранён.

Списки возвращаются страницами. Параметр page выбирает номер страницы, size — количество элементов, а sort — поле и направление сортировки. Адреса `/sort/title` и `/sort/date` задают только сортировку по умолчанию. Если клиент явно передал sort, используется его вариант.

Отдельного запроса GET `/api/v1/reminder/{id}` в сервисе нет. Получить одно напоминание по id через REST сейчас нельзя.

Сквозные сценарии

Создание напоминания

Контроллер проверяет входной JSON и заголовок Idempotency-Key. Затем он берёт `userId` и `email` из JWT и передаёт всё в сервис. Контроллер отвечает за HTTP-формат, но не содержит основную бизнес-логику.

Сервис определяет часовой пояс пользователя и превращает введенный `LocalDateTime` в конкретный момент `Instant`. Напоминание можно создать только на время строго позже текущего момента. Значение, равное «сейчас», тоже отклоняется.

PostgreSQL advisory lock не позволяет одновременно обрабатывать несколько запросов с одинаковыми `userId` и `key`. Первый запрос обрабатывается, остальные ждут. Advisory lock — это временная блокировка внутри транзакции БД; она снимается автоматически после `commit` или `rollback`.

Если запрос с таким `idempotency key` уже успешно выполнялся и для него создан `reminder`, сервис возвращает связанное напоминание. Новый `reminder` не создаётся и ещё одно письмо не планируется. При этом сервис формирует новый ответ из текущего состояния напоминания, а не хранит точную копию первого HTTP-ответа.

Для нового ключа сервис сначала сохраняет `idempotency record` — служебную запись, связывающую пользователя и ключ запроса. Затем он сохраняет `Reminder`, записывает его `id` в служебную запись и создаёт Quartz job для будущего письма.

Эти изменения выполняются внутри JPA-транзакции. Транзакция объединяет несколько изменений БД в одну операцию: при обычном завершении они подтверждаются командой `commit`, а при ошибке отменяются через `rollback`. Ошибка Quartz до завершения метода приводит к откату записей приложения, но код не гарантирует общую атомарность PostgreSQL и Quartz при любом редком сбое.

Частичное обновление

Поле `remind` проверяется только тогда, когда клиент действительно прислал новое время. Если поля нет, прежнее время остаётся.

Сервис снова берёт advisory lock по паре `userId` и `idempotency key`. Запросы с одинаковым ключом выполняются по очереди. После получения блокировки сервис проверяет, не было ли это обновление уже завершено раньше.

PATCH означает частичное обновление. Для каждого непустого поля сервис вызывает `setter` — метод, который меняет конкретное поле объекта. Поля, которых нет в JSON, сохраняют старые значения. Пустой объект `{}` тоже принимается: данные не меняются, но служебная запись `idempotency` создаётся, а Quartz job пересоздаётся.

При изменении напоминания Quartz job пересоздаётся, чтобы в ней оказались новое время и новый текст письма. Сначала старая задача удаляется, затем создаётся новая. Эти два шага не являются одной неделимой операцией.

Удаление

- 1) Сервис ищет Reminder одновременно по reminderId и userId. Если запись принадлежит пользователю, сервис помечает её на удаление и удаляет Quartz job по заранее вычисляемому ключу.
- 2) Если Quartz-задачи уже нет, приложение не считает это причиной отменять удаление Reminder. Оно записывает предупреждение в лог и продолжает работу.
- 3) Idempotency records при удалении Reminder не очищаются. Они остаются до плановой очистки и некоторое время могут ссылаться на уже удалённый объект.

Отправка и повторные попытки

Quartz Job хранит snapshot — копию данных, необходимых для отправки письма на момент создания задачи: email, title и description. Во время запуска задача использует эту копию и не читает Reminder заново из основной таблицы.

Если отправка завершается SMTP-ошибкой, создаётся persisted one-shot trigger. Это сохранённая в PostgreSQL команда Quartz: «запусти эту же задачу ещё один раз в указанное время». Благодаря хранению в БД повтор не теряется после перезапуска приложения.

Между попытками используется exponential backoff — задержка каждый раз удваивается. Формула в коде: $\text{initialDelay} \times 2^{(\text{failedAttempt} - 1)}$. Такая пауза снижает нагрузку на временно недоступный почтовый сервер.

По умолчанию maxAttempts равно 3. Это три попытки всего, а не три повтора: первая выполняется в назначенное время, вторая — через 1 минуту после первой ошибки, третья — через 2 минуты после второй ошибки.

После третьей неудачи новый trigger не создаётся, поэтому цепочка повторов заканчивается. Ошибка остаётся в логах Quartz.

Время

Все моменты времени в сущности Reminder хранятся как Instant — конкретный момент на общей временной шкале, не зависящий от часового пояса.

При поиске напоминаний за определённую дату LocalDate сервис учитывает часовой пояс пользователя. Например, запрос «напоминания за 10 сентября» преобразуется в полуинтервал [начало 10 сентября; начало 11 сентября): начало дня включается, а начало следующего дня уже не входит в результат.

При преобразовании локального времени LocalDateTime в конкретный момент Instant используется TimeZoneResolver. Он проверяет, можно ли однозначно определить offset — смещение местного времени относительно UTC.

При переходе на летнее или зимнее время DST возможны две проблемные ситуации:

- 1) DST gap — указанного локального времени не существовало, потому что часы перевели вперёд.
- 2) DST overlap — указанное локальное время встретилось дважды, потому что часы перевели назад.

TimezoneResolver принимает LocalDateTime только при одном допустимом offset. Если offset нет из-за DST gap или найдено два из-за DST overlap, сервис возвращает ошибку и просит указать другое время.

Каталог рабочего кода

Точка входа

ReminderApplication

Исходник *reminder/src/main/java/dan/reminder/ReminderApplication.java*

Главный класс приложения. Именно с него начинается запуск модуля reminder. Аннотация `SpringBootApplication` также указывает Spring, где искать остальные компоненты проекта.

Методы

```
public static void main( String [] args)
```

Запускает Spring Boot. Spring создаёт общий контекст приложения — набор настроенных объектов, соединение с БД, web-сервер, безопасность, Quartz и остальные зависимости. После этого сервис начинает принимать HTTP-запросы.

Конфигурация

EmailRetryProperties

Исходник *reminder/src/main/java/dan/reminder/config/EmailRetryProperties.java*

Хранит настройки повторной отправки email из `application.yml`. Это `record` — компактный неизменяемый класс данных, который Spring заполняет при запуске.

По умолчанию разрешено 3 попытки всего, а начальная задержка равна `PT1M`. `PT1M` — стандартная запись ISO 8601, означающая одну минуту.

Методы

```
public EmailRetryProperties
```

Проверяет настройки сразу при запуске: `maxAttempts` должен быть от 1 до 10, а `initialDelay` — больше нуля. Если условие нарушено, приложение не запускается с некорректной конфигурацией.

Java автоматически создаёт методы `maxAttempts()` и `initialDelay()` для чтения значений, а также `equals()`, `hashCode()` и `toString()`. Компактный конструктор `record` одновременно является его основным конструктором и выполняет проверку.

IdempotencyProperties

Исходник *reminder/src/main/java/dan/reminder/config/IdempotencyProperties.java*

Хранит настройки защиты от повторных запросов: сколько времени помнить ключ и как часто удалять старые служебные записи.

По умолчанию ключ хранится 24 часа. Первая очистка запускается через 1 минуту после старта, следующие — через 1 час после завершения предыдущей.

Методы

```
public IdempencyProperties
```

Получает три длительности из конфигурации и по очереди проверяет каждую через `requirePositive`.

```
private static void requirePositive(String property, Duration value)
```

Общий вспомогательный метод проверки. Он запрещает отсутствующее, нулевое или отрицательное значение и добавляет имя ошибочной настройки в сообщение.

Java автоматически создаёт три метода чтения значений, основной конструктор, `equals()`, `hashCode()` и `toString()`.

JpaConfig

Исходник `reminder/src/main/java/dan/reminder/config/JpaConfig.java`

Включает JPA auditing — механизм автоматического заполнения служебных полей сущностей. В этом проекте `createdAt` дополнительно и надёжно задаётся самим сервисом, поэтому работа поля не зависит только от auditing.

Обычных методов в классе нет. Spring видит аннотацию `EnableJpaAuditing` при запуске и включает соответствующий механизм.

OpenApiConfig

Исходник `reminder/src/main/java/dan/reminder/config/OpenApiConfig.java`

Настраивает описание API для Swagger: название, версию v1 и схему авторизации Bearer JWT. Swagger использует эти данные, чтобы показать документацию и поле для токена.

Обычных методов нет. Вся настройка записана в аннотациях класса и считывается библиотекой OpenAPI.

SchedulingConfig

Исходник `reminder/src/main/java/dan/reminder/config/SchedulingConfig.java`

Включает методы с аннотацией `Scheduled` и регистрирует два класса настроек. Сам Quartz Spring Boot создаёт автоматически на основании зависимостей и `application.yaml`.

Обычных методов нет: класс служит переключателем конфигурации для Spring.

TimeConfig

Исходник `reminder/src/main/java/dan/reminder/config/TimeConfig.java`

Создаёт единый источник текущего времени. Благодаря этому бизнес-код не обращается к системным часам напрямую, а тест может подставить заранее выбранный момент.

Методы

```
Clock clock()
```

Возвращает bean `Clock.systemUTC`. Bean — объект, который создаёт Spring и передаёт в классы, где нужен `Clock`. Метод доступен внутри пакета; в тестах вместо него можно использовать `Clock.fixed`.

TimezoneResponseHeaderFilter

Исходник `reminder/src/main/java/dan/reminder/config/TimezoneResponseHeaderFilter.java`

HTTP-фильтр, который выполняется один раз для каждого подходящего запроса и сообщает клиенту, когда сервис применил часовой пояс по умолчанию.

Методы

```
protected void doFilterInternal( HttpServletRequest request ,
                                HttpServletResponse response,
                                FilterChain filterChain)
```

Если X-Timezone отсутствует или содержит только пробелы, добавляет в ответ X-Timezone-Applied: Asia/Almaty и X-Timezone-Defaulted: true. Затем всегда передаёт запрос следующему фильтру. Само значение часового пояса здесь не проверяется.

```
protected boolean shouldNotFilter( HttpServletRequest request )
```

Возвращает true для адресов вне `/api/v1/reminder`. Это означает «не запускать данный фильтр» для остальных частей приложения.

Безопасность

PemKeys

Исходник `reminder/src/main/java/dan/reminder/security/PemKeys.java`

Вспомогательный класс, который превращает открытый RSA-ключ из текстового формата PEM в объект `RSAPublicKey`, понятный библиотеке проверки JWT.

Методы

```
private PemKeys( )
```

Закрытый конструктор запрещает создавать объекты `PemKeys`: у класса есть только статические вспомогательные методы.

```
public static RSAPublicKey parsePublicKey( String pem)
```

Убирает из PEM служебные строки и пробелы, декодирует Base64 и создаёт RSA public key. Если формат неверный, выбрасывает `IllegalStateException`, и приложение не сможет настроить проверку JWT.

SecurityConfig

Исходник `reminder/src/main/java/dan/reminder/security/SecurityConfig.java`

Настраивает сервис как OAuth2 Resource Server. Это значит, что `reminder` не выполняет login и не выдаёт токены: он только проверяет JWT, созданные `auth-service`.

Сервис не проверяет роли или `scopes`. Доступ к данным ограничивается иначе: каждый запрос к БД содержит `userId` из JWT, поэтому возвращаются только записи владельца.

Методы

```
SecurityFilterChain security( HttpSecurity http)
```

Отключает CSRF и серверные сессии. Каждый защищённый запрос должен принести JWT заново. Без токена доступны только Swagger, OpenAPI и проверки `health`; остальные адреса требуют успешной аутентификации.

```
Jwt Decoder jwt Decoder( )
```

Создаёт объект проверки JWT. Подпись должна использовать RS256, токен не должен быть просрочен, issuer должен равняться auth-service, а audience должен содержать reminder-rest.

```
private OAuth2TokenValidator<Jwt> audienceValidator( )
```

Возвращает дополнительную проверку поля aud. Audience показывает, для какого сервиса выпущен токен. Если reminder-rest в списке нет, токен отклоняется как invalid_token.

Контроллеры

RemindersRestController

Исходник `reminder/src/main/java/dan/reminder/controller/RemindersRestController.java`

Контроллер для операций со списком напоминаний: создание, поиск, сортировка и демонстрационное заполнение. Контроллер переводит HTTP-запрос в вызов сервиса и формирует HTTP-ответ.

Методы

```
public ResponseEntity<ReminderDto> createReminder( . . . )
```

Обработывает POST /api/v1/reminder. Берёт userId и email из JWT, передаёт тело запроса, часовой пояс и idempotency key в сервис. После создания возвращает статус 201 и заголовок Location с адресом нового объекта.

Аннотация Valid запускает проверку CreateReminderPayload до вызова сервиса. Если обязательное поле отсутствует или строка слишком короткая, клиент получает 400.

Заголовок Idempotency-Key обязателен, не может состоять только из пробелов и должен быть не длиннее 255 символов.

```
public ResponseEntity<PageResponse<ReminderDto>> findAllSortedByTitle( . . . )
```

Возвращает одну страницу напоминаний текущего пользователя. По умолчанию это первая страница из 3 элементов с сортировкой title по убыванию. Явный параметр sort клиента имеет приоритет.

```
public ResponseEntity<PageResponse<ReminderDto>> findAllSortedByDate( . . . )
```

Возвращает страницу напоминаний с сортировкой по времени remind по убыванию, если клиент не указал другую сортировку.

```
public ResponseEntity<PageResponse<ReminderDto>> findRemindersByTitle( . . . )
```

Ищет введённый фрагмент в title без учёта регистра. Поиск всегда ограничен напоминаниями пользователя из JWT.

```
public ResponseEntity<PageResponse<ReminderDto>> findRemindersByDate( . . . )
```

Принимает календарную дату в формате ISO, например 2026-09-10. Сервис вычисляет начало этого дня и начало следующего в часовом поясе пользователя, затем ищет записи внутри полученного диапазона.

```
public ResponseEntity<Page<ReminderDto>> &gt; ;
findRemindersByDateRange( . . . )
```

Принимает start и end как Instant, то есть уже как конкретные моменты времени. Сервис проверяет, что start раньше end. Часовой пояс здесь влияет только на время в ответе, но не меняет границы поиска.

```
public ResponseEntity<String> > ; addSomeReminders( Jwt jwt )
```

Добавляет два случайных демонстрационных напоминания и планирует письма. Метод возвращает обычный текст Reminders added. Защиты Idempotency-Key у этого служебного адреса нет.

Lombok автоматически создаёт конструктор, через который Spring передаёт ReminderService.

ReminderRestController

Исходник `reminder/src/main/java/dan/reminder/controller/ReminderRestController.java`

Контроллер для операций над одним напоминанием: частичного обновления и удаления.

Методы

```
public ResponseEntity<Void> > ; deleteReminder( Long reminderId, Jwt jwt )
```

Обрабатывает DELETE. Получает userId из JWT и просит сервис удалить запись только при совпадении владельца. При успехе возвращает 204 No Content без тела ответа.

```
public ResponseEntity<ReminderDto> > ; updateReminder( . . . )
```

Обрабатывает PATCH. Проверяет частичный JSON и Idempotency-Key, получает userId и email из JWT и возвращает обновлённое напоминание со статусом 200.

Lombok автоматически создаёт конструктор, через который Spring передаёт ReminderService.

CreateReminderPayload

Исходник `reminder/src/main/java/dan/reminder/controller/payload/CreateReminderPayload.java`

Record, описывающий JSON для создания. Title и description обязательны; их длина должна быть 3–255 и 3–4096 символов. Remind обязателен и приходит как LocalDateTime без часового пояса.

Обычных методов в исходнике нет. Ограничения полей записаны аннотациями Bean Validation и проверяются контроллером.

Java автоматически создаёт основной конструктор, методы title(), description() и remind(), а также equals(), hashCode() и toString().

UpdateReminderPayload

Исходник `reminder/src/main/java/dan/reminder/controller/payload/UpdateReminderPayload.java`

Record для частичного обновления. Любое поле можно не присылать: null означает «оставить старое значение». Если строка передана, она должна пройти проверку длины. Поэтому пустой объект {} допустим, а пустая строка для title или description — нет.

Обычных методов в исходнике нет. Аннотации описывают правила проверки переданных значений.

Java автоматически создаёт основной конструктор, методы title(), description() и remind(), а также equals(), hashCode() и toString().

DTO

ReminderDto

Исходник `reminder/src/main/java/dan/reminder/dto/ReminderDto.java`

Объект, который сервис возвращает клиенту. DTO означает Data Transfer Object — отдельную модель для передачи данных через API. В ней есть `id`, `title`, `description`, время со смещением `OffsetDateTime` и `userId`.

Обычных методов в исходнике нет: `record` только хранит данные ответа.

Java автоматически создаёт основной конструктор, пять методов чтения полей, `equals()`, `hashCode()` и `toString()`.

PageResponse

Исходник `reminder/src/main/java/dan/reminder/dto/PageResponse.java`

Собственная форма ответа со страницей данных. Она скрывает внутреннее устройство Spring Data Page и делает JSON-контракт API стабильнее.

Методы

```
public static <T> PageResponse<T> from( Page<T> page )
```

Получает Spring Page и переносит в PageResponse список элементов, номер и размер страницы, общее количество элементов и остальные признаки пагинации. Новый отдельный список не создаётся: используется `content` исходной Page.

Java автоматически создаёт основной конструктор, девять методов чтения полей, `equals()`, `hashCode()` и `toString()`.

Модель данных

Reminder

Исходник `reminder/src/main/java/dan/reminder/model/Reminder.java`

JPA entity для таблицы `reminders`. Entity — Java-объект, который Hibernate связывает с записью в БД. Hibernate читает поля напрямую. После создания `userId` менять нельзя; `title`, `description` и `remind` изменяемы.

Методы

```
private Reminder( String title, String description, Instant remind, UUID userId)
```

Создаёт новое напоминание через Lombok builder. Все четыре значения обязательны. `id` не передаётся: его выдаёт PostgreSQL sequence при сохранении.

```
public void updateContent( String title, String description, Instant remind)
```

Одновременно заменяет `title`, `description` и `remind` и запрещает `null`. Текущий PATCH этот метод не использует: он меняет только переданные поля отдельными `setters`.

```
public boolean equals( Object o)
```

Сравнивает две сохранённые сущности по ненулевому `id` с учётом объектов Hibernate. Два ещё не сохранённых объекта без `id` не считаются равными.

```
public final int hashCode( )
```

После сохранения вычисляет hash по id, до сохранения — по идентичности объекта в памяти. Поэтому новый Reminder не следует помещать в HashSet или использовать ключом HashMap, если затем этому объекту будет назначен id.

```
public String toString( )
```

Возвращает текст с id, title, remind и userId для логов и отладки. Description не включается, чтобы не печатать полный текст напоминания.

Lombok создаёт защищённый пустой конструктор для JPA, методы чтения всех полей, setters для изменяемых полей и builder для удобного создания объекта.

IdempotencyRecord

Исходник *reminder/src/main/java/dan/reminder/model/IdempotencyRecord.java*

JPA entity для служебной таблицы idempotency_record. Она связывает userId и idempotencyKey с id результата и хранит время создания записи. Полный HTTP-ответ и содержимое запроса здесь не сохраняются.

Обычных методов в исходнике нет. createdAt задаёт сервис; аннотация CreatedDate также отмечает поле для JPA auditing.

Lombok создаёт пустой и полный конструкторы, методы чтения и изменения пяти полей и builder.

Репозитории

Оба интерфейса расширяют JpaRepository. Repository — слой доступа к БД. Spring создаёт реализацию во время запуска, поэтому в проекте нет классов с ручным SQL для обычных операций save, find, delete, пагинации и сортировки. Ниже перечислены только методы, добавленные самим проектом.

ReminderRepository

Исходник *reminder/src/main/java/dan/reminder/repository/ReminderRepository.java*

Интерфейс доступа к таблице reminders. Тип Long означает, что первичный ключ Reminder имеет тип Long.

Методы

```
Optional<Reminder> findByIdAndUserId( Long reminderId, UUID userId)
```

Ищет запись сразу по id и userId. Такой запрос одновременно находит напоминание и проверяет владельца, поэтому применяется при PATCH и DELETE.

```
PagedList<Reminder> findAllByUserId( UUID userId, Pageable pageable)
```

Возвращает только напоминания указанного пользователя и применяет переданные параметры страницы и сортировки.

```
PagedList<Reminder> findByTitleContainingIgnoreCaseAndUserId( String title, UUID userId, Pageable pageable)
```

Spring строит запрос по имени метода. Он ищет фрагмент строки title без учёта регистра и обязательно ограничивает результат userId.

```
Pag e& l t ; Reminder& g t ; findByUserIdAndRemindGreat erThanEqual AndRemindLessThan( UUID
userId, Inst ant from, Inst ant t o, Pag eabl e pag eabl e)
```

Ищет записи пользователя в диапазоне [from; to): момент from включён, момент to не включён. Такое правило не создаёт пересечения между соседними днями.

IdempotencyRecordRepository

Исходник *reminder/src/main/java/dan/reminder/repository/IdempotencyRecordRepository.java*

Интерфейс доступа к служебным idempotency records. UUID используется как первичный ключ самой служебной записи.

Методы

```
void acquireTransact ionLock( St ring l ockKey)
```

Просит PostgreSQL установить advisory lock для 64-битного hash строки userId:key. Hash — короткое числовое представление строки. Блокировка действует до commit или rollback текущей транзакции.

```
Opt ional & l t ; Idempot encyRecord& g t ; findByUserIdAndIdempot encyKey( UUID userId,
St ring idempot encyKey)
```

Ищет ранее сохранённый результат операции по пользователю и idempotency key.

```
int del et eCreat edBefore( Inst ant cut off)
```

Одной командой удаляет записи, созданные строго раньше cutoff — рассчитанной границы хранения. Перед удалением Hibernate отправляет накопленные изменения в БД. После операции он очищает свой список отслеживаемых объектов, чтобы в памяти не остались сущности, которых уже нет в таблице.

Сервисы

ReminderService

Исходник *reminder/src/main/java/dan/reminder/service/ReminderService.java*

Интерфейс основной бизнес-логики. Контроллеры зависят от него, а не от конкретного QuartzReminderService. Это позволяет заменить реализацию или подставить mock в тесте.

Методы

```
ReminderDt o creat eReminder( . . . )
```

Создать напоминание с защитой от повторного запроса и поставить задачу отправки письма.

```
void del et eReminder( Long reminderId, UUID userId)
```

Удалить напоминание владельца и связанную задачу Quartz.

```
ReminderDt o updat eReminder( . . . )
```

Изменить только переданные поля с защитой от повторного запроса и пересоздать задачу Quartz.

```
Pag e& l t ; ReminderDt o& g t ; findAl l Sort edByTit l e( . . . )
```

Вернуть страницу напоминаний; если клиент не указал sort, сортировать по title.

```
Pag e& l t ; ReminderDt o& g t ; findAl l Sort edByRemindDat e( . . . )
```

Вернуть страницу напоминаний; если клиент не указал sort, сортировать по времени remind.

```
Pag e& l t ; ReminderDt o& g t ; findRemindersByTit l e( . . . )
```

Найти напоминания пользователя, в title которых встречается указанный текст.

```
Pag e& l t ; ReminderDt o& g t ; findRemindersByDat e( . . . )
```

Найти напоминания, попадающие в выбранный календарный день пользователя.

```
Pag e& l t ; ReminderDt o& g t ; findRemindersByDat eRang e( . . . )
```

Найти напоминания между двумя конкретными моментами времени.

```
void addSomeReminders( UUID userId, St ring userEmail )
```

Добавить демонстрационные данные и запланировать для них письма.

QuartzReminderService

Исходник *reminder/src/main/java/dan/reminder/service/QuartzReminderService.java*

Главная реализация ReminderService. Этот класс связывает HTTP-данные с БД, правилами времени, защитой от повторов и Quartz. Здесь находится большая часть бизнес-логики сервиса.

Методы

```
publ ic ReminderDt o creat eReminder( . . . )
```

Выполняет создание по шагам: проверяет часовой пояс и будущее время, блокирует одинаковый ключ, ищет старый результат, сохраняет служебную запись и Reminder, планирует Quartz job и формирует DTO для ответа.

Важно: повтор определяется только по userId и key. Код не сравнивает новое тело запроса, email и вид операции с первым запросом.

Если Quartz выбрасывает ошибку до завершения метода, исключение не скрывается.

JPA-транзакция откатывает изменения бизнес-таблиц.

```
privat e void acquireIdempot encyLock( UUID userId, St ring idempot encyKey)
```

Собирает строку userId:key и передаёт её репозиторию, который устанавливает PostgreSQL advisory lock.

```
privat e Idempot encyRecord creat eIdempot encyRecord( UUID userId, St ring idempot encyKey)
```

Создаёт служебную запись с текущим временем Clock. reminderId сначала равен null, потому что само напоминание ещё не сохранено.

```
privat e Reminder creat eAndSaveReminder( Creat eReminderPayl oad payl oad, UUID userId, Inst ant remindAt )
```

Создаёт Reminder из проверенного payload, владельца и рассчитанного Instant, затем передаёт его repository.save. Внутри транзакции Hibernate синхронизирует объект с БД.

```
privat e ReminderDt o g et Exist ing ReminderDt o( . . . )
```

Используется при повторном POST. Находит связанный Reminder, пишет событие в лог и создаёт DTO. Новых записей и задач Quartz не появляется.

```
privat e Reminder g et Exist ing Reminder( . . . )
```

Если reminderId в служебной записи равен null, код считает операцию незавершённой или неконсистентной и выбрасывает ошибку. Иначе Reminder ищется по id; дополнительная проверка владельца здесь не выполняется.

```
public void deleteReminder( Long reminderId, UUID userId)
```

Ищет Reminder по паре reminderId и userId, удаляет его и просит Quartz удалить задачу. Возвращаемое Quartz значение true или false не влияет на результат удаления.

```
public ReminderDto updateReminder( . . . )
```

Выполняет PATCH внутри транзакции: проверяет новое время, если оно передано, блокирует idempotency key, ищет прежний результат, меняет только непустые поля, связывает служебную запись с Reminder и пересоздаёт Quartz job.

Пустой PATCH допустим. Поля остаются прежними, но idempotency record и новая Quartz job всё равно создаются.

Повторный ключ не сравнивается с reminderId и содержимым PATCH. Поэтому один key нельзя безопасно использовать для разных напоминаний или разных операций.

```
private ReminderDto getExistingUpdatedReminderDto( . . . )
```

Используется при повторном PATCH. Возвращает текущее состояние Reminder без ещё одного изменения и без повторного планирования письма.

```
private Reminder getExistingUpdatedReminder( . . . )
```

Проверяет, что служебная запись уже содержит reminderId, затем ищет этот Reminder вместе с userId. В отличие от create здесь повторно проверяется владелец.

```
public Page<ReminderDto> findAllSortedByReminderDate( . . . )
```

Метод только для чтения. Передаёт общей функции поле remind как сортировку по умолчанию.

```
public Page<ReminderDto> findAllSortedByTitle( . . . )
```

Метод только для чтения. Передаёт общей функции поле title как сортировку по умолчанию.

```
private Page<ReminderDto> findAllSortedBy( . . . )
```

Готовит сортировку и страницу. Если несколько записей имеют одинаковое основное поле, добавляет id по возрастанию, чтобы порядок был стабильным. Затем проверяет page и size, выбирает часовой пояс и преобразует каждую entity в DTO.

```
public Page<ReminderDto> findRemindersByTitle( . . . )
```

Проверяет page и size. Если сортировка не указана, использует remind, затем id по возрастанию. После этого вызывает repository-поиск по фрагменту title и userId.

```
public Page<ReminderDto> findRemindersByDate( . . . )
```

Вычисляет начало указанного локального дня и начало следующего дня в часовом поясе пользователя. Оба момента переводятся в Instant и передаются методу поиска по диапазону.

```
public Page<ReminderDto> findRemindersByDateRange( . . . )
```

Требует, чтобы from был раньше to. Затем создаёт PageRequest и ищет записи пользователя в диапазоне [from; to), где правая граница не включается.

```
private Instant toFutureInstant( LocalDateTime localDateTime, ZoneId zoneId)
```

Однозначно переводит LocalDateTime в Instant и проверяет, что результат строго позже текущего времени Clock. Иначе выбрасывает InvalidReminderTimeException.

```
private ReminderDto mapToDto( Reminder reminder, ZoneId zoneId)
```

Копирует поля Reminder в ReminderDto. Сохранённый Instant преобразуется в OffsetDateTime выбранного пользователем часового пояса.

```
public void addSomeReminders( UUID userId, String userEmail )
```

Служебный демонстрационный метод. Перемешивает 18 заготовок, выбирает две, назначает время через 2–11 минут, сохраняет их и планирует письма.

Lombok создаёт конструктор со всеми обязательными зависимостями и поле `logger` для записи событий в лог.

ReminderTemplate

Исходник *nested record in QuartzReminderService*

Закрытый `record` внутри `QuartzReminderService`. Он хранит одну заготовку демонстрационного напоминания: `title`, `description` и задержку до отправки в секундах.

Обычных методов в исходнике нет: `record` используется только как контейнер данных.

Java автоматически создаёт основной конструктор, три метода чтения, `equals()`, `hashCode()` и `toString()`.

TimezoneResolver

Исходник *reminder/src/main/java/dan/reminder/service/TimezoneResolver.java*

Компонент для работы с часовыми поясами IANA, например `Asia/Almaty` или `Europe/Berlin`. Он не хранит состояние и безопасно переводит локальное время в `Instant`.

Методы

```
public ZoneId resolveRequired( String timezone)
```

Несмотря на слово `Required` в названии, отсутствие значения или строка из пробелов приводят к `Asia/Almaty`. Непустое значение передаётся внутреннему методу `parse`.

```
public ZoneId resolveOrDefault( String timezone)
```

Работает так же: использует `Asia/Almaty` при пустом значении или разбирает явно указанную зону. Разница между двумя публичными методами сейчас только в названии.

```
public Instant toUnambiguousInstant( LocalDateTime localDateTime, ZoneId zoneId)
```

Получает список допустимых смещений UTC для `LocalDateTime` в выбранной зоне. Если смещение ровно одно, возвращает `Instant`. Ноль или два значения означают DST gap или overlap и приводят к `InvalidReminderTimeException`.

```
private ZoneId parse( String timezone)
```

Вызывает стандартный `ZoneId.of`. Если Java не знает такое название зоны, преобразует техническую `DateTimeException` в понятную для API `InvalidTimezoneException`.

EmailService

Исходник *reminder/src/main/java/dan/reminder/service/EmailService.java*

Небольшой интерфейс отправки email. Основной код зависит от интерфейса, поэтому в тестах настоящий почтовый сервер можно заменить `mock`-объектом.

Методы

```
void sendTextEmail( String to, String subject, String body)
```

Описывает отправку простого текстового письма получателю `to` с темой `subject` и текстом `body`. Метод ничего не возвращает; ошибки во время отправки приходят как `runtime exceptions`.

DefaultEmailService

Исходник *reminder/src/main/java/dan/reminder/service/DefaultEmailService.java*

Рабочая SMTP-реализация EmailService. SMTP — сетевой протокол отправки почты; JavaMailSender скрывает низкоуровневое соединение с сервером.

Методы

```
public void sendText Email ( String to, String subject , String body)
```

Создаёт SimpleMailMessage, заполняет получателя, тему, текст и заранее заданный адрес отправителя. Затем вызывает JavaMailSender.send и пишет успешный результат в лог. Ошибки почтового сервера не перехватываются здесь, чтобы Quartz мог запланировать повтор.

Lombok создаёт конструктор для JavaMailSender и поле logger.

IdempotencyCleanupService

Исходник *reminder/src/main/java/dan/reminder/service/IdempotencyCleanupService.java*

Периодически удаляет старые служебные записи idempotency. Сами напоминания и задачи Quartz этот класс не трогает.

Удаляются только idempotency records; записи reminders и Quartz jobs остаются без изменений.

Методы

```
public int deleteExpiredRecords( )
```

По расписанию вычисляет cutoff: текущее время минус retention. Затем удаляет записи строго старше этой границы, пишет количество удалённых строк в лог и возвращает это число.

Lombok создаёт конструктор с нужными зависимостями и поле logger.

Планировщик и email

QuartzEmailScheduler

Исходник *reminder/src/main/java/dan/reminder/scheduler/quartz/scheduler/QuartzEmailScheduler.java*

Небольшой слой над Quartz Scheduler. Он создаёт, удаляет и пересоздаёт задачи по одинаковым предсказуемым ключам, поэтому для связи с Reminder не нужна отдельная таблица.

Методы

```
public void schedule( Reminder reminder, String userEmail )
```

Копирует email, title, description и номер попытки в JobDataMap — набор данных задачи. Затем создаёт EmailReminderJob и одноразовый trigger на время Reminder.remind. Техническую SchedulerException превращает в IllegalStateException.

```
public boolean delete( Long reminderId)
```

Удаляет задачу email-job-{id}. Если задача найдена, возвращает true. Если её уже нет, пишет предупреждение и возвращает false.

```
public void scheduleRetry( JobDetail jobDetail , int nextAttempt , Duration delay)
```

Создаёт ещё один одnorазовый trigger для существующей job. В trigger записывается номер следующей попытки и время «сейчас плюс delay». Если назначенное время уже прошло из-за задержки сервера, настройка misfire просит Quartz запустить задачу как можно скорее.

```
public void recreate( Reminder reminder, String userEmail )
```

Обновляет задачу в два шага: удаляет старую и создаёт новую. Если создание падает после успешного удаления, напоминание временно остаётся без Quartz job.

```
private JobKey jobKey( Long reminderId)
```

Строит постоянный ключ email-job-{reminderId} в группе email-jobs. По нему задача находится при обновлении и удалении.

```
private TriggerKey triggerKey( Long reminderId)
```

Строит ключ основного trigger email-trigger-{reminderId} в группе email-jobs.

```
private TriggerKey retryTriggerKey( JobKey jobKey, int attempt )
```

Строит уникальный ключ повторного trigger из имени job и номера попытки.

Lombok создаёт конструктор для Scheduler и поле logger.

EmailReminderJob

Исходник *reminder/src/main/java/dan/reminder/scheduler/quartz/job/EmailReminderJob.java*

Код, который Quartz запускает в назначенное время. Он отправляет письмо и при временной ошибке создаёт сохранённую повторную попытку.

Методы

```
public void execute( JobExecutionContext context )
```

Читает объединённый JobDataMap, вызывает EmailService и ловит ошибку отправки. Пока лимит не достигнут, создаёт новый retry trigger. На последней попытке новых запусков не планирует. Немедленный автоматический повтор Quartz отключён: повторы контролирует отдельный trigger.

Данные текущего trigger имеют приоритет над данными JobDetail. Благодаря этому повторная попытка видит свой номер, а не исходное значение 1.

Если не удалось не только отправить письмо, но и запланировать повтор, обе причины сохраняются в одном исключении: ошибка SMTP добавляется как suppressed, то есть как дополнительная связанная ошибка.

```
private int parseAttempt ( String value)
```

Пытается прочитать положительный номер попытки. Если значение отсутствует, не является числом, равно нулю или отрицательное, безопасно возвращает 1.

```
private Duration retryDelay( int failedAttempt )
```

Вычисляет задержку по правилу exponential backoff: initialDelay умножается на 2 для каждой следующей неудачной попытки.

Lombok создаёт конструктор для зависимостей и поле logger.

Ошибки

ApiResponseResponse

Исходник `reminder/src/main/java/dan/reminder/exception/dto/ApiResponseResponse.java`

Единая форма JSON-ошибки. `errorCode` — стабильный машинный код, `message` — текст для человека, `details` — дополнительные значения, например ошибочные поля.

Методы

```
public ApiResponseResponse( String errorCode, String message,
    Map<String, String> details)
```

Сохраняет полученные значения в `final`-полях. `Map details` не копируется, поэтому вызывающий код теоретически может изменить её после создания ответа.

```
public String getErrorCode()
```

Возвращает машинный код, по которому клиент может различать виды ошибок без анализа текста.

```
public String getMessage()
```

Возвращает понятное человеку сообщение об ошибке.

```
public Map<String, String> getDetails()
```

Возвращает карту дополнительных деталей ошибки.

ReminderNotFoundException

Исходник `reminder/src/main/java/dan/reminder/exception/ReminderNotFoundException.java`

Исключение для ситуации, когда `Reminder` не существует или принадлежит другому пользователю. API намеренно не различает эти случаи и возвращает одинаковый 404.

Методы

```
public ReminderNotFoundException( String message)
```

Создаёт runtime exception и передаёт его текст базовому классу `RuntimeException`.

IdempotencyInProgressException

Исходник `reminder/src/main/java/dan/reminder/exception/IdempotencyInProgressException.java`

Исключение означает, что служебная запись `idempotency` есть, но результат ещё не указан. Обычный параллельный запрос обычно ждёт `advisory lock`; поэтому это скорее защита от неконсистентной или вручную созданной записи. API возвращает 409 Conflict.

Методы

```
public IdempotencyInProgressException( String message)
```

Создаёт runtime exception и передаёт диагностический текст базовому классу.

IdempotencyRecordNotFoundException

Исходник `reminder/src/main/java/dan/reminder/exception/IdempotencyRecordNotFoundException.java`

Исключение предусмотрено для внутренней рассинхронизации, когда `idempotency record` указывает на отсутствующий `Reminder`. `GlobalExceptionHandler` скрывает детали и возвращает 500. Текущий `QuartzReminderService` этот тип не выбрасывает.

Методы

```
public IdempotencyRecordNotFoundException( String message)
```

Создаёт runtime exception и передаёт диагностический текст базовому классу.

InvalidTimeZoneException

Исходник *reminder/src/main/java/dan/reminder/exception/InvalidTimeZoneException.java*

Исключение для неизвестного названия часового пояса. Оно сохраняет исходную строку, чтобы handler мог показать проблемное значение в details ответа.

Методы

```
public InvalidTimeZoneException( String message, String timezone)
```

Передаёт сообщение базовому RuntimeException и сохраняет строку timezone.

Lombok автоматически создаёт getTimezone() для чтения сохранённого значения.

InvalidReminderTimeException

Исходник *reminder/src/main/java/dan/reminder/exception/InvalidReminderTimeException.java*

Исключение для некорректного времени напоминания: момент не в будущем либо LocalDateTime попал в DST gap или overlap. Оно сохраняет исходное время и ZoneId для ответа клиенту.

Методы

```
public InvalidReminderTimeException( String message, LocalDateTime localDateTime, ZoneId timezone)
```

Передаёт сообщение базовому RuntimeException и сохраняет LocalDateTime и timezone.

Lombok автоматически создаёт getLocalDateTime() и getTimezone().

InvalidDateRangeException

Исходник *reminder/src/main/java/dan/reminder/exception/InvalidDateRangeException.java*

Исключение для диапазона, в котором start равен end или находится позже end.

Методы

```
public InvalidDateRangeException( Instant start, Instant end)
```

Создаёт сообщение об ошибке и сохраняет обе границы, чтобы вернуть их клиенту.

```
public String getErrorCode()
```

Возвращает стабильный код INVALID_DATE_RANGE.

```
public Instant getStart()
```

Возвращает начало ошибочного диапазона.

```
public Instant getEnd()
```

Возвращает конец ошибочного диапазона.

InvalidPageRequestException

Исходник *reminder/src/main/java/dan/reminder/exception/InvalidPageRequestException.java*

Исключение для некорректных параметров пагинации page и size.

Методы

```
public InvalidPageRequestException( int page, int size)
```

Создаёт диагностическое сообщение и сохраняет полученные page и size.

```
public String get ErrorCode( )
```

Возвращает стабильный код INVALID_PAGE_REQUEST.

```
public int get Page( )
```

Возвращает номер страницы, который вызвал ошибку.

```
public int get Size( )
```

Возвращает размер страницы, который вызвал ошибку.

GlobalExceptionHandler

Исходник *reminder/src/main/java/dan/reminder/exception/GlobalExceptionHandler.java*

Общий обработчик ошибок REST-слоя. Он превращает Java exceptions в одинаковые JSON-ответы, чтобы контроллеры не повторяли один и тот же код.

Методы

```
public ResponseEntity<Y> handleValidationErrors( MethodArgumentNotValidException ex)
```

Собирает ошибки проверки JSON по именам полей. Если для одного поля нарушено несколько правил, все сообщения попадают в один список.

```
private ResponseEntity<Y> buildValidationErrorResponse( Map<String, List<String>> details)
```

Создаёт ответ 400 Bad Request с кодом VALIDATION_ERROR и переданными деталями.

```
public ResponseEntity<Y> handleGeneric( Exception ex)
```

Записывает полную ошибку и stack trace — цепочку вызовов, которая привела к сбою, — в серверный лог. Клиент получает обезличенный 500 без внутренних деталей.

```
public ResponseEntity<Y> handleInvalidPageRequest( InvalidPageRequestException ex)
```

Возвращает 400 и показывает неверные page и size в details.

```
public ResponseEntity<Y> handleInvalidDateRange( InvalidDateRangeException ex)
```

Возвращает 400 и показывает обе границы start и end.

```
public ResponseEntity<Y> handleInvalidTimezone( InvalidTimezoneException ex)
```

Возвращает 400 и помещает значение X-Timezone в details. Null заменяется пустой строкой, потому что Map.of не разрешает null.

```
public ResponseEntity<Y> handleInvalidReminderTime( InvalidReminderTimeException ex)
```

Возвращает 400 с исходным локальным временем remind и названием часового пояса.

```
public ResponseEntity<T> handleHandlerMethodValidation(HandlerMethodValidationException ex)
```

Собирает ошибки параметров метода и вложенных полей в карту «имя параметра → список сообщений».

```
public ResponseEntity<T> handleTypeMismatch(HandlerMethodArgumentTypeMismatchException ex)
```

Возвращает 400 `INVALID_PARAMETER`, когда Spring не может преобразовать значение пути или query-параметра в нужный Java-тип, например строку в Long.

```
public ResponseEntity<T> handleMalformedBody(HandlerMethodArgumentNotReadableException ex)
```

Возвращает 400 `MALFORMED_REQUEST`, если тело нельзя прочитать как допустимый JSON. Детали ошибки парсера клиенту не раскрываются.

```
public ResponseEntity<T> handleMissingRequestParameter(MissingServletRequestParameterException ex)
```

Возвращает 400 `MISSING_REQUEST_PARAMETER`, когда отсутствует обязательный query-параметр.

```
public ResponseEntity<T> handleInvalidSort(PropertyReferenceException ex)
```

Возвращает 400 `INVALID_SORT` и сообщает имя поля, которого нет в модели сортировки.

```
public ResponseEntity<T> handleMissingRequestHeader(MissingRequestHeaderException ex)
```

Возвращает 400 `MISSING_REQUEST_HEADER`, когда нет обязательного HTTP-заголовка, например Idempotency-Key.

```
public ResponseEntity<T> handleReminderNotFound(ReminderNotFoundException ex)
```

Возвращает 404 `REMINDER_NOT_FOUND`. Исходное сообщение исключения не передаётся клиенту, чтобы не раскрывать сведения о чужих id.

```
public ResponseEntity<T> handleIdempotencyInProgress(IdempotencyInProgressException ex)
```

Возвращает 409 `IDEMPOTENCY_IN_PROGRESS`. Код 409 означает конфликт с текущим состоянием операции.

```
public ResponseEntity<T> handleIdempotencyRecordNotFound(IdempotencyRecordNotFoundException ex)
```

Пишет внутреннюю рассинхронизацию в лог и возвращает обычный 500 без технических деталей.

Аннотация `Slf4j` автоматически создаёт поле `logger`.

Данные и конфигурация

Таблицы приложения

Таблица	Что хранит	Что важно знать
reminders	Пользовательские напоминания	id выдаёт sequence; remind хранится как момент с часовым поясом; user_id определяет владельца
idempotency_record	Связь пользователя и ключа с результатом	Пара user_id и key уникальна; reminder_id может быть пустым; старые записи ищутся по created_at
QRTZ tables	Состояние планировщика Quartz	Хранят jobs, triggers, блокировки и служебное состояние, поэтому задачи переживают перезапуск

Главный changelog Liquibase подключает все файлы миграций из каталога releases. Liquibase запоминает выполненные changesets и не применяет один и тот же шаг повторно.

Для reminders созданы индексы по user_id и времени remind. Для title используется расширение pg_trgm и GIN index: они ускоряют поиск по части строки, а не только по полному совпадению.

В таблице idempotency_record поле reminder_id не связано foreign key с reminders, а user_id разрешает null. Поэтому сама схема БД не гарантирует существование связанного Reminder и корректного пользователя.

Таблицы Quartz создаёт Liquibase. Настройка spring.quartz.jdbc.initialize-schema: never запрещает Spring создавать их автоматически.

SQL-файл Quartz начинается с DROP TABLE IF EXISTS. В обычной работе Liquibase выполняет этот changeset один раз. Запускать файл вручную на рабочей БД нельзя: он удалит сохранённые jobs и triggers.

Основные настройки

Группа	Значение по умолчанию	Что это означает
HTTP	порт 8080	На этом порту доступны REST API и Actuator
База данных	localhost:5432/reminder-rest	PostgreSQL для напоминаний, idempotency и Quartz
JPA	UTC и open-in-view false	Работа с entity заканчивается в сервисном слое; контроллер не должен читать ленивые связи
Idempotency	24 часа; очистка через 1 минуту, затем каждый час	Столько времени сервис помнит использованный ключ
Повтор email	3 попытки; первая задержка PT1M	Всего до трёх отправок, задержка после ошибки начинается с одной минуты
Quartz	PostgreSQL, 5 потоков, одна реплика	Задачи сохраняются в БД; текущая настройка не рассчитана на несколько копий сервиса
SMTP	Gmail, порт 587, STARTTLS	Для входа нужен MAIL_PASSWORD; соединение шифруется после подключения
JWT	issuer auth-service; audience reminder-rest	Токен должен быть выпущен auth-service именно для reminder-rest и подписан RS256

Тесты

При подготовке документа команда `mvnw.cmd test` успешно выполнила 32 unit- и MVC-теста без ошибок. Unit-тест проверяет отдельный класс, а MVC-тест — web-слой без настоящей БД. Классы с суффиксом IT являются интеграционными: Maven Failsafe запускает их командой `verify`. Ниже они описаны по исходникам.

TimezoneResolverTest

Исходник `reminder/src/test/java/dan/reminder/service/TimezoneResolverTest.java`

Проверяет правила работы с часовыми поясами: значение по умолчанию, неизвестную IANA-зону и локальное время, которое отсутствует или повторяется при переводе часов.

```
resolverOrDefault_t_shouldUseAlmatyWhenHeaderIsMissing
```

Проверяет, что без заголовка используется Asia/Almaty, а явно указанная корректная зона сохраняется.

```
resolverRequired_shouldRejectUnknownTimezone
```

Проверяет, что неизвестное название зоны приводит к `InvalidTimezoneException`.

```
toUnambiguousInstant_shouldRejectDstGap
```

Проверяет несуществующее локальное время при переводе часов вперёд.

```
toUnambiguousInstant_shouldRejectDstOverlap
```

Проверяет неоднозначное локальное время при переводе часов назад.

IdempotencyCleanupServiceTest

Исходник `reminder/src/test/java/dan/reminder/service/IdempotencyCleanupServiceTest.java`

Проверяет расчёт срока хранения idempotency records и их очистку. Clock зафиксирован, поэтому ожидаемая граница времени не зависит от момента запуска теста.

```
deleteExpiredRecords_shouldDeleteRecordsOlderThanRetention
```

Фиксированный Clock делает текущее время предсказуемым. Тест проверяет рассчитанный cutoff и количество удалённых записей.

```
properties_shouldRejectNonPositiveRetention
```

Подтверждает, что retention, равный нулю, считается ошибкой конфигурации.

PageResponseTest

Исходник `reminder/src/test/java/dan/reminder/dto/PageResponseTest.java`

Проверяет преобразование внутренней Spring Page в публичный PageResponse без потери содержимого и сведений о странице.

```
from_shouldExposeStablePaginationFields
```

Проверяет, что при преобразовании Spring Page не теряются содержимое, номер и размер страницы, общее количество элементов и остальные признаки пагинации.

ReminderRestControllerTest

Исходник `reminder/src/test/java/dan/reminder/controller/ReminderRestControllerTest.java`

Изолированный тест web-слоя удаления. Настоящая БД не используется, а ReminderService заменён mock-объектом, который только записывает ожидаемые вызовы.

```
del et eReminder_ should dRet urnNoContent _ whenRequest IsValid id
```

Проверяет, что userId владельца берётся из JWT, передаётся сервису, а HTTP-ответ имеет статус 204.

```
del et eReminder_ should dRet urnBadRequest _ whenIdIsValid id
```

Проверяет, что строка вместо числового id приводит к 400 и бизнес-сервис вообще не вызывается.

RemindersRestControllerTest

Исходник *reminder/src/test/java/dan/reminder/controller/RemindersRestControllerTest.java*

Изолированные тесты web-слоя создания и чтения. Они проверяют HTTP-контракт, JWT, заголовки, валидацию, пагинацию и ошибки без подключения PostgreSQL.

```
creat eReminder_ should dRet urnCreated _ whenRequest IsValid id
```

Проверяет полный HTTP-контракт: JWT claims, заголовки, JSON-тело, статус 201, Location и ReminderDto в ответе.

```
creat eReminder_ should dRet urnUnauthorized _ whenJwt IsMissing
```

Проверяет, что запрос без JWT получает 401 Unauthorized.

```
creat eReminder_ should dRet urnBadRequest _ whenIdempotencyKeyIsMissing
```

Проверяет, что отсутствие Idempotency-Key приводит к 400.

```
creat eReminder_ should dRet urnBadRequest _ whenIdempotencyKeyIsBlank
```

Проверяет, что ключ из одних пробелов не проходит валидацию.

```
creat eReminder_ should dRet urnValidationDetailed _ whenPayloadIsValid id
```

Проверяет, что ошибки title, description и remind группируются по полям в details JSON-ответа.

```
findAllSortedById _ should dRet urnPage _ whenDefaultPageableIsUsed
```

Проверяет значения страницы по умолчанию: page 0, size 3 и title по убыванию, а также метаданные ответа.

```
findAllSortedById _ should dRet urnBadRequest _ whenPageRequest IsInvalid id
```

Проверяет, что InvalidPageRequestException превращается в единый JSON-ответ со статусом 400.

```
findAllSortedById _ should dUseReminderTimeByDefault
```

Проверяет, что сортировка по умолчанию использует поле remind по убыванию.

```
findAllSortedById _ should dRet urnUnifiedNotFound
```

Проверяет, что внутренний текст ReminderNotFoundException не раскрывается и клиент получает стабильный ответ 404.

QuartzReminderServiceTest

Исходник *reminder/src/test/java/dan/reminder/service/QuartzReminderServiceTest.java*

Unit-тесты основной бизнес-логики. Зависимости заменены mock-объектами, поэтому можно точно проверить порядок вызовов и поведение каждой ветки.

```
setUp
```

Перед каждым тестом создаёт одинаковые `userId`, `email` и `idempotency key`. Это тестовые данные, а не часть `production`-логики.

```
createReminder_shouldCreateRecordReminderAndQuartzJob_whenKeyIsNew
```

Проверяет порядок нового создания: `lock`, `idempotency record`, `Reminder`, связь результата и `Quartz job`.

```
createReminder_shouldReturnExist ing Reminder_ without Creating OrScheduling Again
```

Проверяет, что повтор с тем же ключом возвращает старый результат без новых записей и новой `job`.

```
createReminder_shouldRejectKeyWhoseFirstRequestIsStillInProgress
```

Проверяет защитный случай: у найденного `idempotency record` ещё нет `reminderId`, поэтому сервис выбрасывает `IdempotencyInProgressException`.

```
updateReminder_shouldChangeOnlyProvidedFields_andRecreateQuartzJob
```

Проверяет, что `PATCH` меняет только переданный `title` и пересоздаёт задачу `Quartz`.

```
updateReminder_shouldNotTouchReminder_whenItBelongsToAnotherUser
```

Проверяет, что другой пользователь не может изменить `Reminder` и не запускает сохранение или `Quartz`.

```
deleteReminder_shouldDeleteReminderAndQuartzJob_whenOwnerRequests
```

Проверяет, что владелец удаляет и запись `Reminder`, и `Quartz job`.

```
deleteReminder_shouldThrowException_ without Changing Quartz_ whenReminderIsMissing
```

Проверяет, что при отсутствующем `Reminder` сервис возвращает ошибку до обращения к `Quartz`.

```
findByDateRange_shouldRejectReversedRange
```

Проверяет, что диапазон с `from` позже `to` отклоняется до запроса к `repository`.

```
findByDateRange_shouldUseLocalDayInUserTimezoneAndExclusiveEnd
```

Проверяет перевод календарного дня пользователя в UTC-диапазон с включённым началом и исключённым концом, а также стабильную сортировку.

```
createReminder_shouldRejectUnknownTimezoneBeforeWritingData
```

Проверяет, что неизвестный часовой пояс отклоняется до любых изменений в БД.

```
createReminder_shouldValidateFutureAfterConvertingUserTimezone
```

Проверяет, что правило «время должно быть в будущем» применяется после учёта часового пояса пользователя.

```
createReminder_shouldPropagateQuartzFailure
```

Проверяет, что ошибка `Quartz` не скрывается и доходит до вызывающего кода.

```
privateReminderReminder(...)
```

Вспомогательный метод теста создаёт `Reminder` и через `reflection` задаёт `id`, который в `production` выдаёт БД.

EmailReminderJobTest

Исходник `reminder/src/test/java/dan/reminder/scheduler/quartz/job/EmailReminderJobTest.java`

Проверяет retry-логику без настоящей почты: после первой ошибки создаётся следующая попытка, а после достижения лимита цепочка заканчивается.

```
scheduler.persistedRetryWhenEmailDeliveryFails
```

Имитирует ошибку первой отправки. Проверяет, что создаётся вторая попытка через одну минуту и Quartz получает JobExecutionException.

```
stopSchedulerRetryAfterConfiguredLimit
```

Имитирует ошибку третьей, последней попытки. Проверяет, что следующий trigger уже не создаётся.

RemindersRestControllerIT

Исходник `reminder/src/test/java/dan/reminder/controller/RemindersRestControllerIT.java`

Интеграционные тесты полного пути от HTTP до настоящей PostgreSQL в Testcontainer. Quartz Scheduler заменён mock-объектом, чтобы отдельно проверить данные и команды планировщику.

```
postgreSQLProperties
```

Подставляет в Spring адрес и учётные данные PostgreSQL, который Testcontainers временно запускает в Docker.

```
cleanDatabase
```

Перед каждым тестом удаляет служебные записи, затем reminders, чтобы тесты не влияли друг на друга.

```
createReminderShouldCreateReminderInDatabaseWhenRequestIsValidId
```

Проверяет путь от HTTP-запроса до настоящей PostgreSQL: создаются Reminder и idempotency record, а scheduler получает команду создать job.

```
createReminderShouldReturnUnauthorizedWhenUserIsNotAuthenticated
```

Проверяет, что запрос без аутентификации получает 401 и ничего не меняет в БД или Quartz.

```
findAllSortedByIdShouldReturnFirstPageSortedByIdDescendingWhenDefaultPageableIsUsed
```

Проверяет страницу и сортировку по умолчанию на настоящих записях PostgreSQL.

```
findAllSortedByIdShouldUseRequestPageSizeAndSortWhenPageableParametersAreProvided
```

Проверяет, что явно переданные size и sort действительно меняют результат запроса.

```
createReminderShouldReturnExistenceResultWhenRequestIsRepeatedWithSameKey
```

Дважды отправляет запрос с одним ключом и подтверждает, что появляется только один Reminder и одна Quartz job.

```
createReminderShouldRollbackDataWhenQuartzSchedulerFails
```

Имитирует ошибку scheduler и проверяет rollback обеих таблиц приложения. Этот тест не доказывает полную атомарность JPA и Quartz при любом сбое.

```
findAllSortedByIdShouldReturnOnlyCurrentUserReminders
```

Создаёт данные двух пользователей и проверяет, что каждый видит только свои напоминания.

```
privateReminder saveReminder(...)
```

Вспомогательный метод создаёт тестовый Reminder в БД.

ReminderRestControllerIT

Исходник `reminder/src/test/java/dan/reminder/controller/ReminderRestControllerIT.java`

Интеграционные тесты PATCH и DELETE с настоящей БД. Они проверяют владельца, 404, частичное обновление и обязательный idempotency key.

```
post g resProperties
```

Подставляет настройки временной PostgreSQL из Testcontainers.

```
cleanDatabase
```

Очищает таблицы перед тестом, чтобы результат не зависел от предыдущих запусков.

```
deleteReminder_shouldDeleteReminder_whenOwnerRequests
```

Проверяет, что владелец получает 204 и запись действительно исчезает из БД.

```
deleteReminder_shouldReturnNotFound_whenReminderDoesNotExist
```

Проверяет ответ 404 для несуществующего id.

```
updateReminder_shouldUpdateReminder_whenOwnerRequests
```

Проверяет изменение title, description и remind владельцем.

```
updateReminder_shouldReturnNotFound_whenUserIsNotOwner
```

Проверяет, что другой пользователь получает 404 и запись остаётся без изменений.

```
updateReminder_shouldPreserveOmittedFields_whenPatchIsPartial
```

Проверяет семантику PATCH: поля, которых нет в JSON, сохраняют прежние значения.

```
updateReminder_shouldReturnBadRequest_whenIdempotencyKeyIsMissing
```

Проверяет, что без Idempotency-Key запрос получает 400 и title в БД не меняется.

QuartzJdbcIntegrationTest

Исходник `reminder/src/test/java/dan/reminder/scheduler/QuartzJdbcIntegrationTest.java`

Проверяет реальный Quartz JDBC JobStore: job и trigger действительно записываются в PostgreSQL и удаляются вместе с Reminder в обычном сценарии.

```
post g resProperties
```

Подставляет реальную временную PostgreSQL для бизнес-таблиц и Quartz JDBC JobStore.

```
cleanState
```

Очищает Quartz и таблицы приложения перед каждым тестом.

```
createAndDeleteReminder_shouldUpdateBusinessAndQuartzTablesTogether
```

Проверяет согласованное поведение в обычном сценарии: создание добавляет Reminder, job и trigger, а удаление убирает все три записи.

QuartzEmailRetryIT

Исходник `reminder/src/test/java/dan/reminder/scheduler/QuartzEmailRetryIT.java`

Сквозной тест повторной отправки. PostgreSQL хранит Quartz trigger, а GreenMail изображает SMTP-сервер: первая попытка падает, после восстановления почты повтор доставляет письмо.

```
postgreSQL resProperties
```

Подставляет PostgreSQL Testcontainer для хранения Quartz jobs и triggers.

```
cleanStart
```

Останавливает тестовый SMTP-сервер и очищает Quartz и таблицы приложения, чтобы подготовить начальную ошибку отправки.

```
stopSMTPServer
```

После каждого теста останавливает GreenMail — небольшой тестовый SMTP-сервер.

```
healthProbesArePublicAndRetryDeliversThroughSMTPAfterServerRecovers
```

Проверяет полный сценарий: health доступен без JWT, первая отправка падает, retry trigger сохраняется в PostgreSQL, после запуска SMTP повтор доставляет письмо.

```
private boolean waitUntil(Duration timeout, BooleanSupplier condition)
```

До истечения timeout периодически проверяет условие. Возвращает true, как только условие выполнено.

```
private boolean hasRetryTrigger(JobKey jobKey)
```

Просматривает triggers указанной job и ищет имя с окончанием retry-2, то есть вторую попытку.

Запуск и диагностика

Локальный запуск

```
cd D:\project s\reminder\reminder
$env:MAIL_PASSWORD = "&lt; ; gmail - app- password&gt; ; "
. \mvnw.cmd spring - boot :run
```

Для запуска нужен PostgreSQL с БД reminder-rest на порту 5432. Можно запустить весь набор сервисов командой docker compose up --build из корня репозитория: она поднимет PostgreSQL, reminder, auth-service и reminder-client.

Проверки

```
. \mvnw.cmd test # быстрые unit - и MVC- тесты
. \mvnw.cmd verify # дополнительно интеграционные тесты с Test containers
```

Swagger UI доступен по адресу <http://localhost:8080/swagger-ui/index.html>. Через него можно посмотреть REST-методы и выполнить запрос с JWT.

Проверки состояния: /actuator/health, /actuator/health/liveness и /actuator/health/readiness. Liveness отвечает, работает ли процесс; readiness — готов ли сервис принимать трафик, включая доступность БД.

Метрики /actuator/metrics и /actuator/prometheus требуют JWT.

Dockerfile собирает jar с флагом skipTests. Поэтому автоматическая сборка должна запускать тесты отдельным шагом до создания образа.

Ограничения и технический долг

Изменение Reminder в JPA и изменение Quartz выполняются двумя разными механизмами. В проекте нет outbox — отдельной таблицы событий, которая позволила бы надёжно довести планирование задачи после сохранения данных. Нет и явно настроенной общей транзакции. При редком сбое одна часть может сохраниться, а другая — нет; тогда появится Quartz job без Reminder или Reminder без правильной job.

QuartzEmailScheduler.recreate сначала удаляет старую job, а затем создаёт новую. Если второй шаг падает, напоминание остаётся без задачи отправки.

Idempotency record хранит только userId, key, reminderId и время. В нём нет HTTP-метода, адреса, целевого id, hash тела запроса или копии ответа. Поэтому повтор другого запроса с тем же ключом может вернуть результат первой операции.

После удаления Reminder старая служебная запись idempotency остаётся до плановой очистки. Повтор запроса по такому ключу может попытаться найти уже отсутствующий объект.

Доставка email работает по принципу at least once — письмо должно быть отправлено хотя бы один раз, но при редком сбое возможен дубликат. Отдельного журнала доставок и защиты от повторного письма нет.

Quartz настроен для одной копии приложения. Перед запуском нескольких реплик нужно включить clustering, уникальные instanceId и общий JobStore, иначе планировщики могут работать несогласованно.

Адрес отправителя в Java-коде и SMTP username в application.yaml привязаны к конкретному почтовому адресу. Их лучше вынести в настройки окружения.

Проверка JWT ожидает UUID в sub и email в claim email, но эти два условия отдельно не валидируются. Некорректный sub или отсутствующий email могут привести к внутренней ошибке 500.

Нет отдельных тестов для PemKeys, реальной проверки RSA-подписи JWT, TimezoneResponseHeaderFilter и OpenAPIConfig. MVC-тесты с тестовым JWT не проверяют настоящий public key, issuer и audience.

Демонстрационный адрес addsomereminders доступен любому пользователю с корректным JWT. Он не использует Idempotency-Key, поэтому повторный вызов создаёт новые данные.

Первый безопасный маршрут по коду

Начните с RemindersRestController и ReminderRestController. Они показывают доступные HTTP-операции, обязательные заголовки и данные, которые берутся из JWT.

Затем откройте ReminderService как список возможностей и QuartzReminderService как их реализацию. Читайте методы группами: create, update, read, delete и demo.

Для логики времени сопоставьте TimezoneResolver с тестами DST. Так проще увидеть не только успешное преобразование, но и два отклоняемых случая.

Для хранения данных сравните три уровня: JPA entity описывает Java-объект, repository выполняет запросы, а Liquibase migrations создают реальные таблицы и индексы.

Для отправки письма пройдите цепочку QuartzEmailScheduler → EmailReminderJob → DefaultEmailService. Первый класс планирует, второй выполняет и повторяет, третий обращается к SMTP.

Перед изменением idempotency прочитайте QuartzReminderServiceTest и интеграционные тесты обоих контроллеров. Они фиксируют поведение при новом и повторном ключе, ошибке Quartz и чужом Reminder.

Перед изменением retry прочитайте EmailReminderJobTest и QuartzEmailRetryIT. Первый изолированно проверяет расчёт повторов, второй показывает полный путь через PostgreSQL и тестовый SMTP.